

# Turing Machine: Generator

CS-414: Theoretical Computer Science

Online Class – November 22, 2024

# Recap

TM as an acceptor, for a set  $S$  of words over  $\Sigma$ ,

- Recursively Enumerable Sets – There is a TM which accepts every word in  $S$  and either rejects or loops for every word in  $\Sigma^* - S$ .

Note -A finite state machine with two pushdown stores is said to be equivalent to TM.

- Recursive Set – There is a TM which accepts every word in  $S$  and rejects every word in  $\Sigma^* - S$ .

## Question?

1. Is every Recursive set also Recursively enumerable set?
2. Is a *class of Recursive sets*  $\subseteq$  *class of Recursively Enumerable Set* ? Or
3. Is a *class of Recursive sets*  $\subset$  *class of Recursively Enumerable Set* ?

- If a set over  $\Sigma$  is recursive, its complement is \_\_\_\_\_
- If a set over  $\Sigma$  is recursive, its complement is Recursive.
- \*There are recursively enumerable sets of words over  $\Sigma$  such as that their complement is not recursively enumerable.
- A set  $S$  over  $\Sigma$  is recursive if and only if both  $S$  and its complement  $\Sigma^* - S$  is recursively enumerable.

# TMs with Output

- In a multi-tape TM, A tape in which no overwriting is permitted.

In effect the writing may be done only in an empty cell of the one-tape TM or one of the tapes in a multi-tape TM only writing is done and moving to the right no left movement of the tape head.

- A TM to perform concatenation,

Let  $w, x, y \in L$  over an alphabet  $\Sigma$ . Can a TM with  $w = xy$  be achieved by input of  $x \bullet y$ ?

|   |   |   |   |   |   |   |   |   |  |  |  |
|---|---|---|---|---|---|---|---|---|--|--|--|
| 0 | 0 | 1 | 0 | • | 1 | 1 | 1 | 0 |  |  |  |
| 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 |   |  |  |  |

# Church's Hypothesis /Church-Turing Thesis

The intuitive notion of ***computable functions can be identified with the class of partial recursive functions.***

Consider a class of  $n$ -ary functions ( $n \geq 1$ ) Turing Computable functions, mapping  $n$ -tuples of words over  $\Sigma$  into words over  $\Sigma$ . [to include exactly all computable  $n$ -ary partial functions mapping  $n$ -tuples of words over  $\Sigma$  into a word over  $\Sigma$ .]

Partial functions indicate that  $f(w_1, w_2, \dots, w_n)$  may be undefined for some  $n$ -tuples of words  $(w_1, w_2, \dots, w_n)$  over  $\Sigma$  in contrast to total functions for which  $f(w_1, w_2, \dots, w_n)$  is defined and yields a word over  $\Sigma$  for all possible  $n$ -tuples of words over  $\Sigma$ .

# Partial Functions

A partial function  $f(x_1, x_2, \dots, x_n)$ ,  $n \geq 1$ , mapping  $n$ -tuples of words over  $\Sigma = \{a, b\}$  into words over  $\Sigma$  is said to be Turing computable if there is a Turing machine  $M$  over  $\{a, b, \bullet\}$  which behaves as follows:

For every  $n$ -tuple  $(w_1, w_2, \dots, w_n)$  of words over  $\Sigma$   $M$ , for the input of the strings  $w_1 \bullet w_2 \bullet \dots \bullet w_n$ ,

1. If  $f(x_1, x_2, \dots, x_n)$  is undefined, then  $M$  will loop forever.
2. If  $f(x_1, x_2, \dots, x_n)$  is defined, then  $M$  will eventually halt (either rejecting or accepting the input) with the tap containing the value of  $f(x_1, x_2, \dots, x_n)$  followed by empty/blanks.

Input tape

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| a | a | a | • | b | b | b | Δ | Δ | Δ |
|---|---|---|---|---|---|---|---|---|---|

Output Tape

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| a | a | a | b | b | b | Δ | Δ | Δ | Δ |
|---|---|---|---|---|---|---|---|---|---|

# Primitive Recursive Functions

Every  $n$ -ary ( $n \geq 1$ ) primitive recursive function is a total function mapping  $n$ -tuple of words over  $\Sigma$  into words over  $\Sigma$ .

The class of all primitive recursive functions over the alphabet  $\{a, b\}$  is defined as follows:

1. Base functions:

$$\text{Nil}(x) = \Delta$$

$$\text{Consa}(x) = ax$$

$$\text{Consb}(x) = bx$$

## 2. Composition

If  $h, g_1, g_2, \dots, g_m$  are primitive recursive functions over  $\Sigma$ , then the  $n$ -ary function  $f$  is defined by,

$$f(x_1, x_2, \dots, x_n) = h(g_1(x_1, x_2, \dots, x_n), g_2(x_1, x_2, \dots, x_n), \dots, g_m(x_1, x_2, \dots, x_n))$$

## 3. Primitive recursion:

If  $g, h_1, h_2$  are primitive recursive functions over  $\Sigma$ , then so is the  $n$ -ary function  $f$  defined as

If  $n = 1$

$$f(\Delta) = w \quad [\text{any word } w \text{ over } \Sigma^*]$$

$$f(ax) = h_1(x, f(x))$$

$$f(bx) = h_2(x, f(x))$$



If  $n \geq 2$

$$f(\Delta, x_2, \dots, x_n) = g(x_2, \dots, x_n)$$

$$f(ax_1, x_2, \dots, x_n) = h_1(x_1, x_2, \dots, x_n, f(x_1, x_2, \dots, x_n))$$

$$f(bx_1, x_2, \dots, x_n) = h_2(x_1, x_2, \dots, x_n, f(x_1, x_2, \dots, x_n))$$

Exercise,

$$\text{nil}(x) = ?$$

$$\text{consa}(\text{nil}(x)) = ?$$

$$a(x) = \text{consa}(\text{nil}(x))$$

$$a(x) = a \text{ ???}$$

# Turing Machine as an Algorithm

- Problem, Decidable, Undecidable
- Question – Is a given CFG ambiguous?  
An instance of a problem is a list of arguments, one argument for each parameter of the problem, with Yes/No answer.
- A problem whose language is recursive is said to be *decidable*. Otherwise the problem is *undecidable*.

# Yes/No Problems

- Halting Problem of Turing Machines – Unsolvable
- Partial solvability of classes of Yes/no Problems
  - Halting problem of Turing machine is partially solvable.
  - For some instances of the problem is defined and for some not.
  - There is an algorithm that takes any Turing machine  $M$  over  $\{a, b\}$  and any word  $w \in \{a, b\}^*$  as input and
    1. If  $M$  halts, the algorithm will eventually reach an Accept halt, but
    2. If  $M$  does not halt for  $w$ , the algorithm either reaches a Reject state or never halts.

So,  $P$  is solvable if and only if both  $P$  and  $\Sigma^* - P$  are partially solvable.

Class  $P$  of Yes/No problems is reducible to  $P$  and  $P$  is (not) partially solvable